

Projeto Final em Engenharia Informática

Musical Theory Trainer

COM GERAÇÃO PROCEDIMENTAL E AVALIAÇÃO AUTOMÁTICA

RELATÓRIO INTERMÉDIO

João Paulo Ramos Ferreira, aluno 1800238

Orientador: Professor Pedro Duarte Pestana

3 de maio de 2026

Índice

Capítulo 1 - Introdução	5
1.1. Descrição do Projeto	5
1.2. Âmbito e Enquadramento.....	5
1.3. Necessidades Identificadas	5
1.4. Objetivos.....	6
1.5. O que se pretende fazer	6
1.6. Restrições.....	6
1.7. Resultados Esperados.....	7
1.8. Calendário Geral.....	7
1.9. Estrutura do Relatório.....	8
Capítulo 2 - Desenho	8
2.1. Tecnologias e Ferramentas	8
2.2. Arquitetura do Sistema	9
2.3. Estrutura de Dados.....	11
2.4. Algoritmos para a Geração Procedimental.....	11
2.5. Justificação de Opções e Alternativas.....	12
2.6. Interface do Utilizador	13
Capítulo 3 - Implementação	17
3.1. Estrutura de Decomposição do Trabalho (WBS).....	17
3.2. Precedências e Distribuição de Tarefas	18
3.3. Calendarização e Progresso	18
3.4. Relatório de Estado da Implementação.....	18
Bibliografia	19

Lista de Figuras

Figura 1 - Diagrama C4 Nível 1: Contexto do Sistema.....	9
Figura 2 - Diagrama C4 Nível 2: Contentores do Sistema	10
Figura 3 - Diagrama de Sequência: Ciclo de Geração de Exercício	10
Figura 4 - Modelo de Dados: Diagrama de Classes UML	11
Figura 5 - Ecrã Inicial	13
Figura 6 - Ecrã de apresentação	14
Figura 7 - Exemplo de exercício para classificação de intervalos.....	14
Figura 8 - Exemplo de exercício para classificação de intervalos - resposta errada	15
Figura 9 - Exemplo de exercício para classificação de intervalos - resposta correta.....	15
Figura 10 - Exemplo de exercício para identificação de escalas	16
Figura 11 - Exemplo de exercício para identificação de escalas - resposta errada.....	16
Figura 12 - Exemplo de exercício para identificação de escalas - resposta correta.....	17

Lista de Tabelas

Tabela 1 - Cronograma geral do projeto	7
Tabela 2 - Decisões de arquitetura e alternativas consideradas.....	12
Tabela 3 - Calendarização e estado das tarefas.....	18

Resumo

O *Musical Theory Trainer* (MTT) é uma plataforma web interativa para apoio ao estudo autónomo da teoria musical, desenvolvida no âmbito da U.C. *Projeto Final em Engenharia Informática* da Universidade Aberta. A plataforma gera procedimentalmente exercícios para identificação de intervalos, escalas e tonalidades, combinando a visualização da notação musical em pauta com a reprodução do áudio sintetizado em tempo real, sem recurso a ficheiros pré-gravados. A avaliação das respostas é automática, com visualização imediata e registo persistente do histórico de desempenho do utilizador.

A arquitetura assenta numa *Single Page Application* (SPA) em *JavaScript Vanilla*, comunicando via *API REST* com o *Backend* em *Python (FastAPI)*. A notação musical é gerada em formato vetorial (SVG) pela biblioteca *VexFlow*, a síntese de áudio é assegurada pelo *Tone.js*, e a persistência de dados é garantida por uma base de dados *SQLite* gerida através do ORM *SQLAlchemy*. O motor de geração procedimental implementa um algoritmo de ortografia diatónica que assegura a correção teórica e ortográfica das pautas geradas.

À data do relatório intercalar, os requisitos funcionais nucleares do MVP encontram-se implementados: geração de exercícios de intervalos e escalas, representação visual em pauta, reprodução de áudio, avaliação automática de respostas por escolha múltipla e monitorização de progresso através de um *dashboard* de métricas. A implementação dos exercícios de tonalidades e da justificação pedagógica detalhada está prevista para a fase seguinte do desenvolvimento.

Não se reproduz em anexo ao presente relatório o código do projeto, uma vez que está disponível via GitHub: <https://github.com/jopaferreira/PEI-MTA>.

Capítulo 1 - Introdução

Este capítulo apresenta o projeto *Musical Theory Trainer*. Contém a descrição do problema que pretende resolver e os objetivos fixados para o Produto Mínimo Viável (MVP). Descreve as restrições tecnológicas e apresenta a calendarização das tarefas.

1.1. DESCRIÇÃO DO PROJETO

O estudo da teoria musical requer prática regular e beneficia significativamente da avaliação para a consolidação de conceitos. O *Musical Theory Trainer* – MTT pretende ser uma plataforma web interativa, baseada em geração procedimental, desenhada para apoiar o estudo autónomo de estudantes de música e autodidatas. Ao contrário dos métodos tradicionais em papel — limitados pela ausência de reprodução sonora, esta solução combina a visualização em notação musical (pauta) com áudio sintetizado. São gerados exercícios para identificação de intervalos, escalas e tonalidades, devolvendo a avaliação automática das respostas e esclarecimentos teóricos sempre que a resposta indicada não seja a correta.

1.2. ÂMBITO E ENQUADRAMENTO

O projeto enquadra-se no domínio do desenvolvimento de ferramentas de e-learning para a aprendizagem musical. Trata-se de uma *Single Page Application* (SPA) com o processamento visual e sonoro realizado pelo cliente para garantir uma interface fluida visando proporcionar uma experiência de utilização agradável. A arquitetura do sistema assenta num *Frontend* em *JavaScript Vanilla* e num *Backend* em *Python (FastAPI)*, apoiado por uma base de dados *SQLite* para o armazenamento de métricas.

1.3. NECESSIDADES IDENTIFICADAS

A análise a produtos de referência revelou lacunas que este projeto visa colmatar:

- Ausência de persistência gratuita: ferramentas como o *Musictheory.net* não permitem a gravação de progresso em versões web gratuitas.
- Interfaces obsoletas: plataformas como o *Teoria.com* apresentam curvas de aprendizagem exigentes e design pouco responsivo.

- Ausência de resposta imediata: o estudo tradicional não permite a correção instantânea de erros durante o treino autónomo.

1.4. OBJETIVOS

Para satisfazer as necessidades identificadas, foram definidos os seguintes objetivos:

- Geração procedimental: desenvolver um motor capaz de calcular notas e escalas em tempo real, sem recurso a exercícios estáticos que poderiam ser repetitivos.
- Notação e áudio: desenhar pautas musicais com rigor e reproduzir o som correspondente sem recurso a ficheiros áudio pré-gravados.
- Avaliação automática: validar as respostas do utilizador, em formato escolha múltipla e fornecer justificações teóricas em caso de erro.
- Monitorização de progresso: registar o histórico de desempenho do utilizador, disponibilizando métricas de sucesso/insucesso.

1.5. O QUE SE PRETENDE FAZER

Pretende-se implementar o projeto com base numa arquitetura com três componentes principais:

- *Backend*: implementação da lógica musical e das rotas de API com recurso ao *framework FastAPI*.
- *Frontend*: construção de uma SPA que integra as bibliotecas *VexFlow* para visualização de notação musical e *Tone.js* para síntese de áudio.
- Base de dados: gestão de utilizadores e tentativas através do *ORM SQLAlchemy*, com persistência em *SQLite*.

1.6. RESTRIÇÕES

O desenvolvimento teve em consideração as seguintes restrições técnicas e de segurança:

- Políticas de áudio: os browsers modernos bloqueiam a reprodução automática, exigindo uma interação explícita do utilizador para iniciar o contexto de áudio.
- Ortografia musical: o motor procedimental deve assegurar o uso correto de acidentes (sustenidos e bemóis).
- Escalabilidade no MVP: a adoção de *SQLite* circunscreve o sistema a acessos locais ou de baixa concorrência, não sendo adequada para ambientes de produção de maior escala.

1.7. RESULTADOS ESPERADOS

O resultado esperado é uma aplicação funcional que permita ao utilizador, num único ecrã, gerar uma nova questão, ouvir a reprodução sonora do exercício, submeter a resposta e obter a taxa de resposta corretas atualizada de forma instantânea. A aferição do sucesso do MVP traduz-se na implementação completa e na integração eficaz dos requisitos funcionais nucleares (*Must-have*), nomeadamente:

- a geração procedimental correta dos desafios musicais;
- a sua respetiva representação visual e reprodução sonora sem falhas;
- a avaliação automática das respostas submetidas;
- registo persistente do histórico de desempenho do utilizador.

O critério de desempenho para o objetivo estabelecido é um tempo de resposta da API inferior a um segundo por pedido, garantindo a fluidez do ciclo de interação do utilizador.

1.8. CALENDÁRIO GERAL

O cronograma detalhado do projeto é apresentado na tabela seguinte:

Tabela 1 - Cronograma geral do projeto

Semanas	Conteúdo planeado	Marco
Sem. 1-4	Proposta, Requisitos e Arquitetura	Proposta (25 mar)
Sem. 5-7	<i>Wireframes</i> e Implementação do Núcleo (API/SPA)	Demo interna
Sem. 8	Redação do Relatório Intermédio	Intercalar (6 mai)

Semanas	Conteúdo planeado	Marco
Sem. 9–12	Implementação de Exercícios e Áudio	
Sem. 13–14	Testes funcionais e Polimento da Interface	
Sem. 15–16	Conclusões e Entrega Final	Final (24 jun)

1.9. ESTRUTURA DO RELATÓRIO

O presente relatório encontra-se organizado da seguinte forma:

- Capítulo 1 – Introdução: presente capítulo.
- Capítulo 2 – Desenho: explicita as tecnologias adotadas, apresenta os diagramas de arquitetura e as decisões de interface do utilizador.
- Capítulo 3 – Implementação: descreve a decomposição de tarefas e o estado atual do desenvolvimento.

O relatório final incluirá ainda o Capítulo 4 (Testes) e o Capítulo 5 (Conclusões).

Capítulo 2 - Desenho

Este capítulo explicita a arquitetura técnica do sistema *Musical Theory Trainer*, justificando as escolhas tecnológicas. Apresenta a modelação de dados, a lógica algorítmica e o desenho da interface.

2.1. TECNOLOGIAS E FERRAMENTAS

A *stack* tecnológica foi escolhida para garantir uma aplicação leve, rápida e de fácil manutenção:

- Linguagens e *Frameworks*: *Python 3.11* com a *framework FastAPI* para o desenvolvimento do servidor de API, garantindo um bom desempenho e a geração automática de documentação. No lado do cliente, optou-se por *HTML5*, *CSS3* e *JavaScript Vanilla*, para a implementação do modelo de *Single Page Application* (SPA).
- Bibliotecas de Terceiros:
 - *VexFlow*: Biblioteca para a representação da notação musical em formato *SVG* (*Scalable Vector Graphics*);
 - *Tone.js*: *Framework* para síntese de áudio;
 - *SQLAlchemy*: ORM (*Object-Relational Mapper*) para a gestão da base de dados através de classes *Python* (Fowler, 2002);

- *Pydantic*: Utilizada pelo *FastAPI* para a validação dos dados à entrada da API.

2.2.ARQUITETURA DO SISTEMA

Utiliza-se uma arquitetura cliente-servidor desacoplada. O *Frontend* funciona como contentor *Web App*, que comunica via pedidos HTTP (JSON) com o contentor *API* do *Backend*.

- *Frontend* (SPA): Gere o estado da interface, trata da interação com o utilizador, processa o desenho das pautas e assegura a reprodução do som localmente, libertando o servidor para evitar cargas excessivas.
- *Backend* (API REST): Responsável pelo motor de geração musical procedimental e pela persistência de dados.
- Base de Dados: Armazenamento local em SQLite, que dispensa configuração adicional e garante a portabilidade do projeto.



Figura 1 - Diagrama C4 Nível 1: Contexto do Sistema

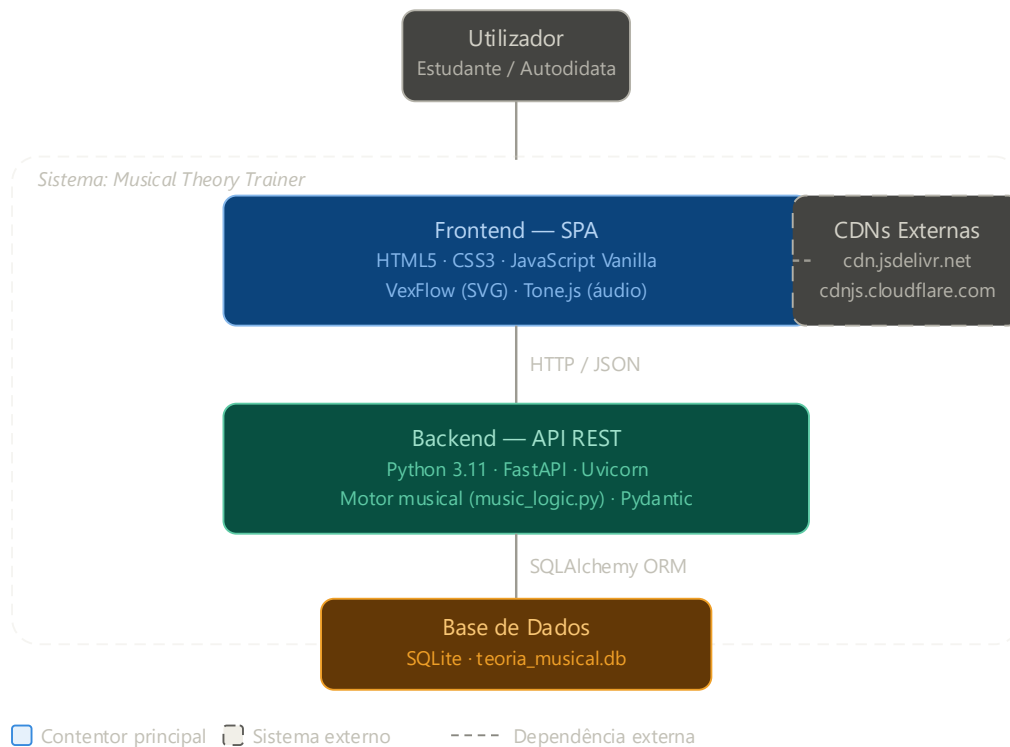


Figura 2 - Diagrama C4 Nível 2: Contentores do Sistema

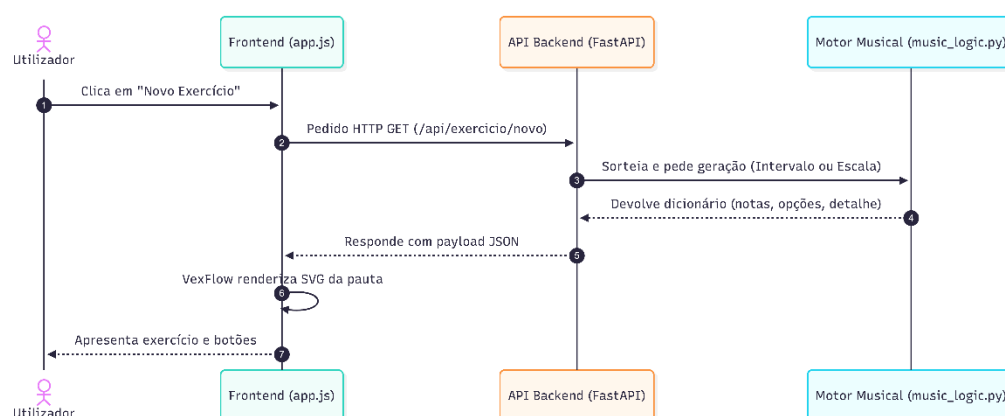


Figura 3 - Diagrama de Sequência: Ciclo de Geração de Exercício

2.3. ESTRUTURA DE DADOS

A base de dados foi modelada para suportar a gestão de tentativas e métricas simples:

- Tabela utilizadores: Armazena o perfil do aluno (id, username, data_registo).
- Tabela tentativas: Regista cada exercício resolvido (id, utilizador_id, tipo_exercicio, detalhe, resposta_dada, correta, data_hora).
- Relacionamento: Existe uma relação de "um para muitos" entre utilizadores e tentativas, permitindo calcular o progresso individual.

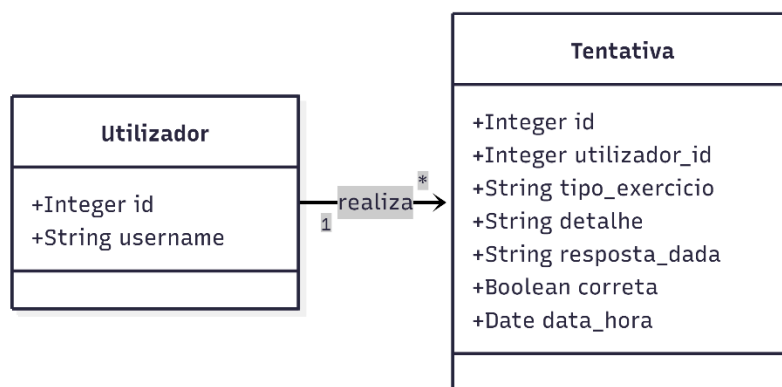


Figura 4 - Modelo de Dados: Diagrama de Classes UML

Na implementação atual do MVP, o sistema opera com um único perfil de utilizador fixo ("Visitante"), criado automaticamente na primeira interação. A tabela *utilizadores* foi modelada para suportar múltiplos perfis, ficando a autenticação individual prevista como requisito *Should-Have* (RF10) para uma fase posterior.

2.4. ALGORITMOS PARA A GERAÇÃO PROCEDIMENTAL

Utiliza-se um algoritmo de Ortografia Diatónica¹ para evitar erros ortográficos musicais (ex: apresentar um Lá Sustenido em vez de um Si Bemol, quando teoricamente incorreto).

- Dicionário de Mapeamento: Mapeia o som, utilizando meios tons, para a nota correta, considerando a letra do alfabeto musical correspondente.

¹ Princípio da teoria musical que estabelece que cada nota de uma escala ou tonalidade deve ser representada por um nome de nota distinto, respeitando a sequência das sete letras musicais (Kostka, Payne & Almén, 2018).

- Lógica de Intervalos: Calcula uma nota somando os meios tons e os saltos necessários a partir de uma nota base aleatória.
- Lógica de Escalas: Aplica padrões de intervalos (ex: Escala Maior [0, 2, 4, 5, 7, 9, 11, 12]) para gerar sequências de notas completas.

2.5.JUSTIFICAÇÃO DE OPÇÕES E ALTERNATIVAS

As decisões foram tomadas com base nas exigências de um MVP, conforme se explicita na tabela seguinte:

Tabela 2 - Decisões de arquitetura e alternativas consideradas

Opção Tomada	Justificação	Alternativa Rejeitada
<i>Python/FastAPI</i>	Sintaxe robusta para algoritmos matemáticos e validação nativa de dados via <i>Pydantic</i> . Geração automática de documentação interativa (<i>Swagger UP</i>).	Node.js / Express: adequado para I/O assíncrono mas com menor expressividade para algoritmos de geração procedimental.
<i>SQLite + SQLAlchemy</i>	Base de dados embebida que dispensa instalação de infraestrutura adicional, facilitando a distribuição e execução do projeto em qualquer máquina. O uso do ORM permite migrar para PostgreSQL com alteração mínima de código no futuro.	PostgreSQL / MySQL: complexidade de configuração desnecessária para a fase de MVP. Ficheiros JSON: sem suporte a <i>queries</i> complexas nem escalabilidade mínima.
SPA com <i>VexFlow</i> e <i>Tone.js</i>	O processamento visual e do áudio é executado no cliente, libertando o servidor. O <i>VexFlow</i> gera notação vetorial (SVG) precisa e escalável. O <i>Tone.js</i> sintetiza áudio em tempo real, eliminando a necessidade de ficheiros pré-gravados.	Aplicação nativa (<i>iOS/Android</i>): fora do âmbito do MVP e limitaria o acesso universal via browser. Imagens e áudio estáticos (PNG/MP3): incompatíveis com o requisito de geração procedimental.
Geração Procedimental	Variedade virtualmente ilimitada de exercícios a cada interação, sem repetição de conteúdos. Garante a correção	Exercícios estáticos em base de dados: exigiria a criação manual de centenas de combinações, tornando o projeto difícil de escalar e manter.

² <http://127.0.0.1:8000/docs>

Opção Tomada	Justificação	Alternativa Rejeitada
	ortográfica e teórica das pautas geradas.	Geração por soma de meios tons sem controlo alfabético: produziria erros de ortografia diatónica.

2.6. INTERFACE DO UTILIZADOR

A *interface* foi desenhada para ser limpa e intuitiva. Apresentam-se algumas capturas de ecrã, que evidenciam o desenho da *interface*.

- *Splash Screen*: Ecrã de carregamento animado com um temporizador de 2 segundos para garantir que os recursos (VexFlow/Tone.js) estão prontos.
- Ecrã Principal: Contentor que apresenta a pauta musical (SVG) e as opções de resposta em formato de escolha múltipla, recorrendo a botões.
- Visualização: Utilização de cores com significado para validação imediata (verde para correto, vermelho para incorreto).
- *Dashboard*: Painel inferior que apresenta o total de respostas e a taxa de acerto global em tempo real.

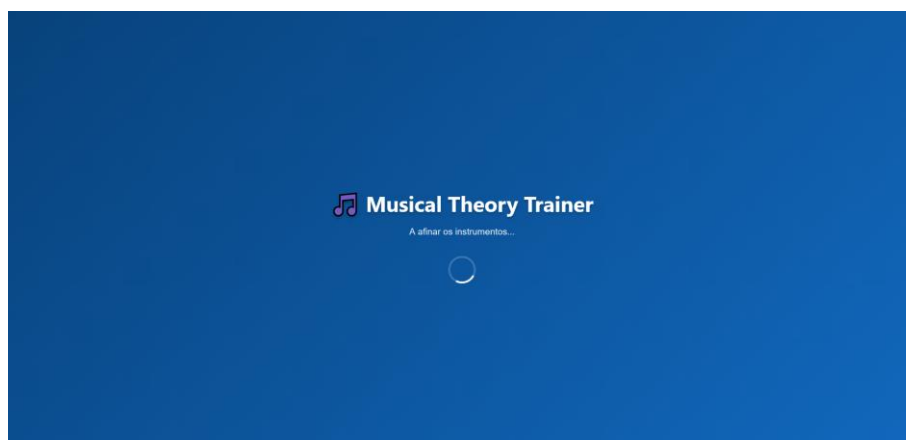


Figura 5 - Ecrã Inicial

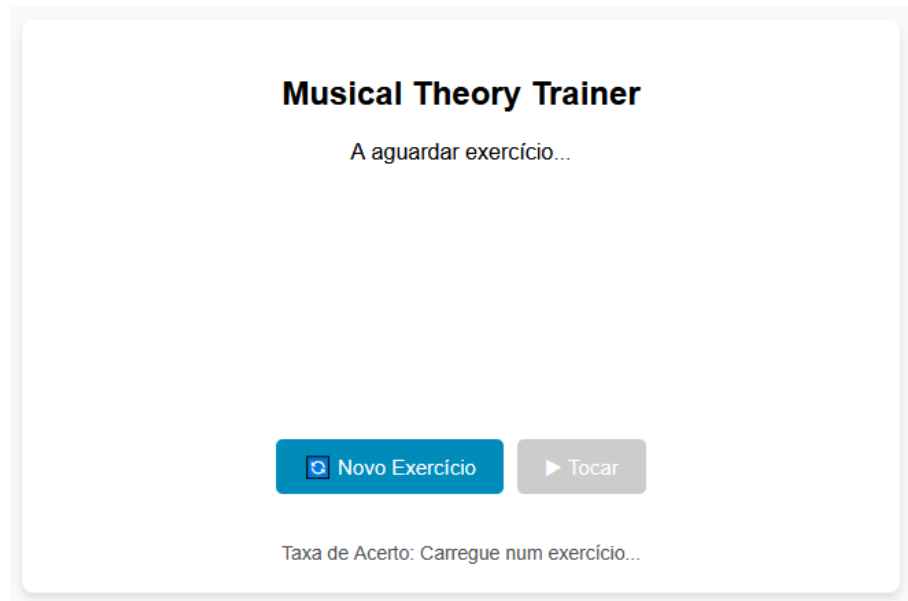


Figura 6 - Ecrã de apresentação

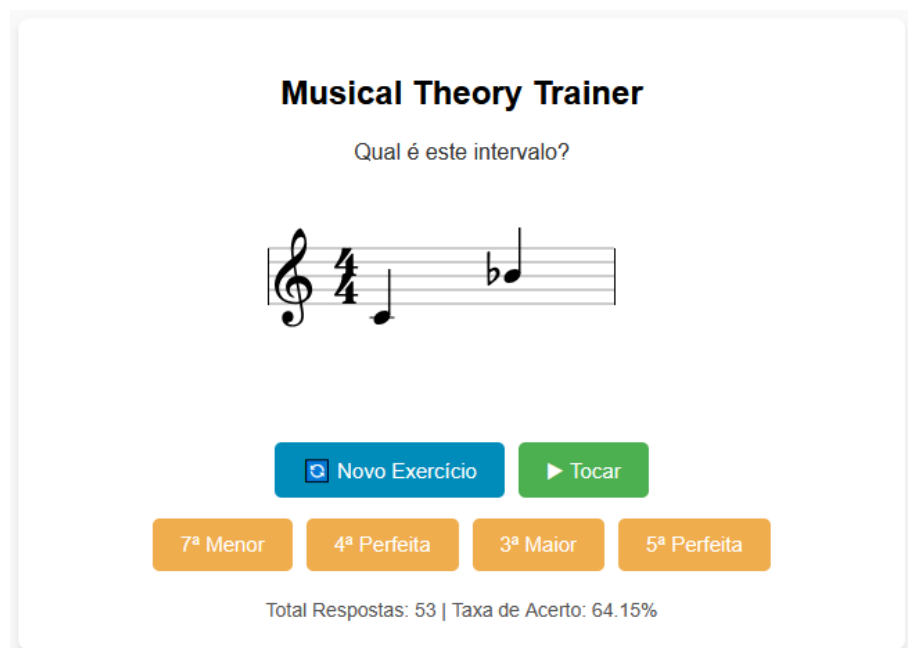
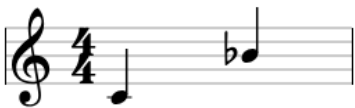


Figura 7 - Exemplo de exercício para classificação de intervalos

Musical Theory Trainer

✖ Errado! A resposta certa era: 7ª Menor.



Novo Exercício Tocar

7ª Menor 4ª Perfeita 3ª Maior 5ª Perfeita

Total Respostas: 54 | Taxa de Acerto: 62.96%

Figura 8 - Exemplo de exercício para classificação de intervalos - resposta errada

Musical Theory Trainer

🌟 Resposta Correta!



Novo Exercício Tocar

4ª Perfeita 6ª Menor 7ª Menor 3ª Menor

Total Respostas: 55 | Taxa de Acerto: 63.64%

Figura 9 - Exemplo de exercício para classificação de intervalos - resposta correta

Musical Theory Trainer

Qual é esta escala?



Novo Exercício Tocar

Escala Menor Harmônica Escala Menor Natural Escala Maior

Total Respostas: 55 | Taxa de Acerto: 63.64%

Figura 10 - Exemplo de exercício para identificação de escalas

Musical Theory Trainer

✗ Errado! A resposta certa era: Escala Maior.



Novo Exercício Tocar

Escala Menor Harmônica Escala Menor Natural Escala Maior

Total Respostas: 56 | Taxa de Acerto: 62.5%

Figura 11 - Exemplo de exercício para identificação de escalas - resposta errada

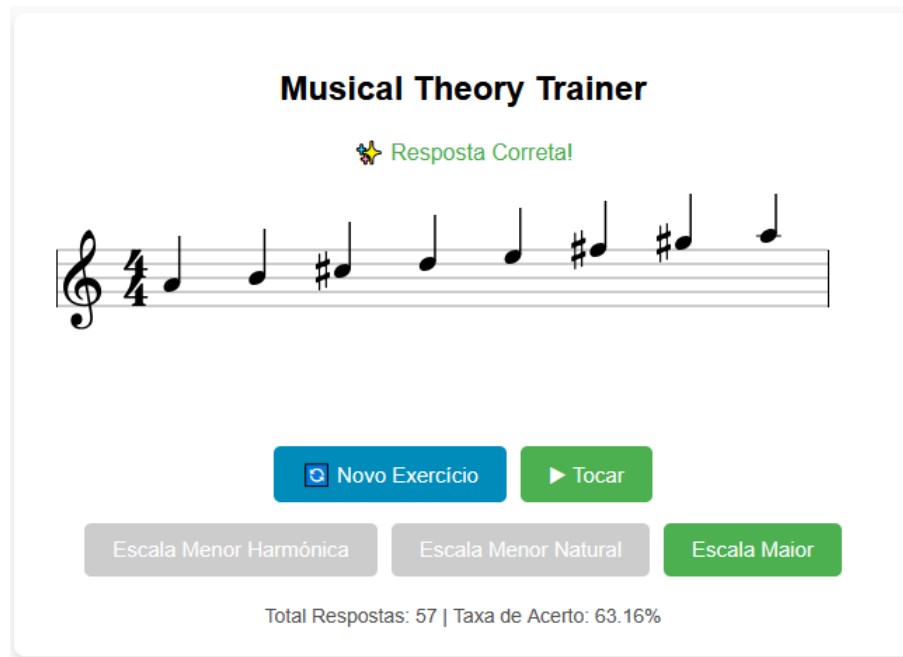


Figura 12 - Exemplo de exercício para identificação de escalas - resposta correta

Capítulo 3 - Implementação

Este capítulo descreve a organização do trabalho em tarefas, a gestão de dependências e o estado atual de desenvolvimento do projeto *Musical Theory Trainer*.

3.1. ESTRUTURA DE DECOMPOSIÇÃO DO TRABALHO (WBS)

Para garantir o cumprimento dos requisitos definidos (*MoSCoW*), o projeto foi decomposto em quatro eixos principais de tarefas:

1. Gestão e Planeamento: Redação da proposta, levantamento de requisitos, gestão de riscos e planeamento de arquitetura.
2. *Backend* (Núcleo Lógico): Implementação da API REST, motor de geração procedimental musical e configuração da base de dados.
3. *Frontend* (Interface e Experiência): Criação da SPA, integração das bibliotecas de visualização (*VexFlow*) e áudio (*Tone.js*), e desenho da UI/UX.
4. Documentação e Relatórios: Manutenção do diário de bordo (*Changelog*), redação de ADRs e elaboração do relatório intermédio.

3.2.PRECEDÊNCIAS E DISTRIBUIÇÃO DE TAREFAS

Tratando-se de um projeto individual, todas as tarefas foram executadas pelo aluno. A ordem de implementação obedece à seguinte lógica de dependências:

1. O motor de lógica musical (*music_logic.py*) teve de ser validado antes da criação das rotas da API, para garantir a correção dos dados enviados.
2. A estrutura do *Backend* e os modelos de dados foram finalizados antes do desenvolvimento do *Frontend*, para permitir o consumo de dados reais.
3. A integração do áudio (*Tone.js*) foi a última tarefa técnica do núcleo, dependendo da interação física do utilizador com os elementos já desenhados no ecrã.

3.3.CALENDARIZAÇÃO E PROGRESSO

O calendário previsto foi respeitado, encontrando-se atualmente na transição entre a fase de consolidação e o início da implementação de funcionalidades secundárias.

Tabela 3 - Calendarização e estado das tarefas

Semanas	Tarefa / Marco	Estado
Sem. 1-2	Proposta e Configuração do Repositório	Concluído
Sem. 3-4	Arquitetura C4, Requisitos e Setup API/BD	Concluído
Sem. 5-6	Lógica de Intervalos/Escalas e Integração VexFlow	Concluído
Sem. 7	Demo Interna e Consolidação de Código	Concluído
Sem. 8	Entrega do Relatório Intermédio	Em curso
Sem. 9-12	Implementação de Tonalidades e Polimento Áudio	Planeado

3.4.RELATÓRIO DE ESTADO DA IMPLEMENTAÇÃO

Nesta data, o estado de implementação dos Requisitos Funcionais (*Must-Have*) é o seguinte:

- RFo1 (Geração de Exercícios): Parcialmente concluído. O motor gera exercícios de intervalos e escalas com ortografia diatónica correta. A geração de exercícios de tonalidades está prevista para as semanas 9-10.
- RFo2 (Representação da Pauta): Concluído. Integração bem-sucedida com *VexFlow*, incluindo gestão e desenho correto dos acidentes musicais (sustenidos, bemóis e bequados).

- RFo3 (Reprodução do áudio): Concluído. Sintetizador implementado via *Tone.js*, com reprodução sequencial das notas geradas, ativado por interação explícita do utilizador para cumprimento das políticas de *autoplay* dos browsers modernos.
- RFo4 (Opções de Escolha Múltipla): Concluído. O sistema apresenta três ou quatro opções de resposta geradas pelo *Backend*, das quais uma corresponde à resposta correta.
- RFo5 (Bloqueio após Submissão): Concluído. Após a seleção de uma opção, todos os botões de resposta são desativados no *Frontend*, impedindo múltiplas submissões para o mesmo exercício.
- RFo6/RFo7 (Avaliação e Feedback Visual): Concluído. A validação da resposta é realizada no *Frontend* por comparação direta com a resposta correta devolvida pelo *Backend*. O resultado é comunicado ao utilizador através de sinalização visual com cores (verde para correto, vermelho para incorreto). Em caso de erro, é apresentada a resposta certa. A justificação pedagógica detalhada está prevista para as semanas 9-10.
- RFo8/RFo9 (Persistência e *Dashboard*): Concluído. O resultado de cada tentativa é enviado ao *Backend* e gravado automaticamente em *SQLite*. O *dashboard* apresenta em tempo real o total de respostas e a taxa de acerto global do utilizador.

É de registar uma limitação conhecida na implementação atual: embora a estrutura de dados suporte múltiplos utilizadores, o sistema não dispõe ainda de autenticação. Todas as tentativas são gravadas sob um perfil único ("Visitante", ID 1), criado automaticamente pelo *Backend* caso não exista. As métricas apresentadas no *dashboard* refletem, por isso, o histórico acumulado de todas as sessões no dispositivo. Esta limitação é consistente com o âmbito do MVP e está identificada como RFI0 (*Should-Have*) no levantamento de requisitos.

Bibliografia

FastAPI. (2026). FastAPI framework, high performance, easy to learn, fast to code, ready for production. Disponível em <https://fastapi.tiangolo.com/> (acedido em 3 de maio de 2026)

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Kostka, S., Payne, D., & Almén, B. (2018). Tonal harmony (8.^a ed.). McGraw-Hill.

Product Plan. (2026). MoSCoW prioritization. Disponível em <https://www.productplan.com/glossary/moscow-prioritization/> (acedido em 3 de maio de 2026)

Pydantic. (2026). Pydantic — data validation using Python type hints. Disponível em <https://docs.pydantic.dev/> (acedido em 3 de maio de 2026)

Simon, S. (2023). The C4 model for visualising software architecture. Disponível em <https://c4model.com/> (acedido em 3 de maio de 2026)

SQLAlchemy. (2026). SQLAlchemy — the Python SQL toolkit and object relational mapper. Disponível em <https://www.sqlalchemy.org/> (acedido em 3 de maio de 2026)

SQLite. (2026). SQLite home page. Disponível em <https://www.sqlite.org/> (acedido em 3 de maio de 2026)

Tone.js. (2026). A Web Audio framework for making interactive music in the browser. Disponível em <https://tonejs.github.io/> (acedido em 3 de maio de 2026)

VexFlow. (2026). VexFlow — music engraving in HTML5. Disponível em <https://vexflow.com/> (acedido em 3 de maio de 2026)